

B.P.Poddar Institute Of Management & Technology

ACADEMIC YEAR: 2026 – 2027

Information Technology

Subject Code-PEC-IT501B

AI PROJECT REPORT

AI PUZZLE SOLVER

A project report submitted in partial fulfilment of the requirements
for the subject Artificial Intelligence

Submitted by

Name :	Roll No :
Mikki Kumari	11500224006
Sachin Kumar	11500224019
Anandi Kumari	11500224021
Rahul Kumar Yadav	11500224022

Under the guidance of
Udayan Misra

CERTIFICATE

This is to certify that the project report entitled “AI-Puzzle Solver” is a bonafide work carried out by Mikki Kumari,Sachin Kumar, Anandi Kumari,Rahul Kumar Yadav, bearing Roll Number 11500224006,11500224019,11500224021,11500224022, in partial fulfilment of the requirements for the subject Artificial Intelligence during the academic year 2026–2027.

The work presented in this report has been completed under my guidance. To the best of my knowledge, the work is suitable for submission and evaluation.

Udayan Misra	Sabnam Sengupta
Project Guide / Faculty	Head of Department

Date: 03-11-2026

Place: Kolkata

DECLARATION

I, Mikki Kumari, Sachin Kumar,Anandi Kumari, Rahul Kumar Yadav, hereby declare that the project entitled “AI-Puzzle Solver” submitted for the subject Artificial Intelligence is my own work carried out under the guidance of UDAYAN MISRA.

I further declare that this report has not been submitted, either in full or in part, to any other institution for the award of any degree, diploma, or certificate. All external sources used for understanding the concepts and preparing this report have been acknowledged in the References section.

Place: Kolkata

Date:03-11-2026

Signature of Student: Mikki Kumari, Sachin Kumar, Anandi Kumari, Rahul Kumar Yadav

ACKNOWLEDGEMENT

I express my sincere gratitude to UDAYAN MISRA for valuable guidance, encouragement, and feedback throughout this AI-project. The guidance helped me understand how search algorithms can be applied to a well-defined problem and how to convert an algorithm into a working software application.

I thank the Head of the Department, the faculty members, and B.P.Poddar Institute of Management & Technology for providing the academic environment and resources required to complete this work. I am also grateful to my classmates, friends, and family for their support.

Finally, I acknowledge the Python documentation and standard-library references that helped me use Python, Tkinter, and the heap-based priority queue.

ABSTRACT

The 8-Puzzle is a classic Artificial Intelligence problem consisting of a 3×3 board with eight numbered tiles and one empty cell. The objective is to transform an initial arrangement into a specified goal arrangement by sliding a tile into the empty cell. Although the rules are simple, the number of possible arrangements makes uninformed trial-and-error search inefficient.

This project develops a desktop 8-Puzzle Solver using Python and the Tkinter graphical user interface. The problem is represented as a state space in which each state is a tuple of nine values. The legal actions move the blank cell Up, Down, Left, or Right whenever the movement remains inside the board. A* Search is used to find an optimal path. Its evaluation function is f(n) = g(n) + h(n), where g(n) is the number of moves already taken and h(n) is the Manhattan Distance heuristic.

The application validates user input, checks puzzle solvability using inversion parity, computes a solution path, and allows the user to inspect the result step by step or through automatic playback. It also displays performance measures such as solution length, expanded states, generated states, and execution time. The project demonstrates state-space modelling, informed search, heuristic evaluation, priority queues, and GUI-based implementation.

Keywords: Artificial Intelligence, 8-Puzzle, A* Search, State Space, Manhattan Distance, Heuristic Search, Python, Tkinter.

1. INTRODUCTION

Artificial Intelligence (AI) is the field of computing concerned with building systems that perform tasks requiring capabilities such as reasoning, planning, learning, perception, and problem solving. Search is a fundamental AI technique. A search problem describes a set of possible states, a starting state, a goal condition, and actions that transform one state into another.

The 8-Puzzle is widely used to explain search because it has a small, clearly defined environment while still exhibiting a large state space. A player slides numbered tiles into a neighbouring empty cell. Only one tile moves at a time, and every move has a fixed cost. The solver must discover a sequence of legal moves that reaches the goal.

This project implements an automated solver rather than relying on a human to choose each move. The user supplies a valid board configuration and the program searches for a path to the standard goal. The graphical interface makes the state transitions visible and provides a practical demonstration of the search process.

The project focuses on a deterministic, fully observable, discrete, single-agent environment. The state and legal actions are known, and the same action applied to the same state always produces the same successor state.

2. PROBLEM STATEMENT

Given a 3×3 board containing the values 1 through 8 and one blank cell represented by 0, find a sequence of legal blank-cell movements that transforms a user-provided initial state into the goal state (1, 2, 3, 4, 5, 6, 7, 8, 0). The solution should use an AI search strategy and should avoid unnecessary moves.

Problem formulation

- Initial state: any valid, solvable permutation of the values 0 to 8.
- Goal state: (1, 2, 3, 4, 5, 6, 7, 8, 0).
- Actions: move the blank Up, Down, Left, or Right if the destination cell exists.
- Transition model: swap the blank with the adjacent tile in the selected direction.
- Step cost: 1 for every legal move.
- Path cost: total number of moves from the initial state to the goal.

The application must also reject malformed input and identify unsolvable configurations before performing an unnecessary search.

3. OBJECTIVES

1. Study state-space representation and problem formulation in Artificial Intelligence.
2. Represent the 8-Puzzle board as a compact data structure.
3. Define the initial state, goal state, legal actions, transition model, and path cost.
4. Implement A* Search using a priority queue.
5. Implement Manhattan Distance as an admissible heuristic.
6. Check the solvability of a 3×3 puzzle using inversion parity.
7. Display the solution path in an interactive Tkinter interface.
8. Measure solution length, expanded states, generated states, and execution time.
9. Test valid, invalid, solved, and unsolvable puzzle configurations.

Scope

The project is limited to the standard 3×3 8-Puzzle and a fixed goal arrangement. It demonstrates deterministic search and does not train a machine-learning model. The same ideas can later be extended to larger puzzles or alternative heuristics.

4. LITERATURE REVIEW / EXISTING SYSTEM

Classical AI literature describes problem solving as search through a graph of states. Breadth-First Search explores states in increasing depth and is complete for finite branching with unit step costs, but it may expand many states. Depth-First Search uses less memory in some cases but does not generally guarantee a shortest solution. Greedy Best-First Search uses a heuristic to select promising states, but it is not generally optimal.

A* Search combines the path cost already incurred with a heuristic estimate of the remaining cost. When the heuristic is admissible, A* can return an optimal solution under the usual positive-cost assumptions. For the 8-Puzzle, Manhattan Distance is a common heuristic: for every numbered tile, calculate the number of horizontal and vertical grid steps between its current position and its goal position, then sum these distances.

Limitations of manual / basic approaches

- Manual solving depends on the player's experience and may involve repeated moves.
- Blind search does not use information about how close a state is to the goal.
- A simple random-move program may loop or fail to reach the goal efficiently.
- A solver without input validation may accept impossible or malformed boards.

The proposed system addresses these issues through an informed search algorithm, a solvability check, and a visual interface.

5. PROPOSED SYSTEM

The proposed system accepts a 9-value puzzle configuration, validates that each digit from 0 to 8 appears exactly once, and checks whether the state is solvable. If the state is solvable, A* explores reachable configurations and returns an optimal path to the goal. The interface displays the board and allows the user to move forward, move backward, or play the solution automatically.

Main modules

- Input and validation module: reads and validates the initial configuration.
- Solvability module: counts inversions and checks parity.
- State transition module: generates all legal successor states.
- Heuristic module: calculates Manhattan Distance.
- A* search module: manages the frontier, costs, and parent links.
- Path reconstruction module: rebuilds the ordered state and move sequences.
- GUI module: displays the board, controls, status, and performance metrics.

The program is modular: algorithm.py contains the search logic and main.py contains the Tkinter interface. This separation makes the algorithm easier to test independently of the user interface.

6. AI TECHNIQUE USED

A* Search

A* is an informed best-first search algorithm. For each discovered state n, it calculates f(n) = g(n) + h(n). The frontier is maintained as a priority queue, so the state with the smallest estimated total path cost is selected first.

- g(n): exact cost from the initial state to n; here it is the number of moves.
- h(n): estimated cost from n to the goal; here it is Manhattan Distance.
- f(n): estimated total cost of a solution path passing through n.

Manhattan Distance

For a numbered tile i, let (r_i, c_i) be its current row and column and (r_i*, c_i*) be its goal row and column. Its Manhattan contribution is |r_i − r_i*| + |c_i − c_i*|. The heuristic is the sum of the contributions for tiles 1 through 8. The blank is excluded.

Manhattan Distance is admissible because a tile must move at least its row difference plus its column difference to reach its target, and one legal move shifts only one numbered tile by one grid position. It is also consistent for the unit-cost 8-Puzzle. Therefore A* with this heuristic finds a shortest solution.

Solvability

An inversion is a pair of numbered tiles that appear in the opposite relative order from ascending order when the blank is removed. For a 3×3 puzzle, a state is solvable relative to the standard goal exactly when its inversion count is even. This parity test avoids searching states that cannot reach the goal.

7. SYSTEM ARCHITECTURE / BLOCK DIAGRAM

The system follows the pipeline below:

1.User enters the initial 3×3 configuration.
2. Input validator checks length, range, and uniqueness.
3. Solvability checker evaluates inversion parity.
4. A* search initializes the priority queue with the initial state.
5. The search expands legal neighbours and calculates g, h, and f.
6. When the goal is selected, parent links are used to reconstruct the path.
7. The GUI displays the sequence and search-performance information.

Module interaction

The GUI calls validate_state() and is_solvable() before calling solve_astar(). The algorithm module returns a result dictionary containing path, moves, cost, expanded, and generated values. The GUI uses the path to render each board state. Execution time is measured around the search call using Python's high-resolution performance counter.

Suggested block diagram for final formatting:

User Input → Validation → Solvability Check → A* Search (Priority Queue + Manhattan Distance) → Path Reconstruction → Tkinter Output.

8. ALGORITHM / PSEUDOCODE

A* procedure

```
A_STAR(start, goal):
    if start is not solvable:
        return FAILURE
    frontier = priority queue ordered by f = g + h
    g_score[start] = 0
    parent[start] = NONE
    insert start with priority h(start)

    while frontier is not empty:
        current = remove state with minimum priority
        if current is goal:
            return reconstruct_path(parent, current)

        for each legal neighbour of current:
            tentative_g = g_score[current] + 1
            if tentative_g is better than known cost:
                g_score[neighbour] = tentative_g
                parent[neighbour] = (current, action)
                priority = tentative_g + Manhattan(neighbour)
                insert neighbour with priority

    return FAILURE
```

Neighbour generation

Find the blank index and convert it into row = index // 3 and column = index % 3. Up is legal when row > 0; Down when row < 2; Left when column > 0; Right when column < 2. For each legal action, swap the blank with the destination tile to create a successor state.

Path reconstruction follows parent references from the goal to the initial state, then reverses the collected states and actions. This avoids storing a complete path in every frontier entry.

9. IMPLEMENTATION

Development environment

- Language: Python 3.9 or later.
- GUI toolkit: Tkinter from the Python standard library.
- Priority queue: heapq.
- Timing: time.perf_counter().
- Testing: unittest.

Data representation

Each state is an immutable tuple of nine integers. Tuple states can be used as dictionary keys and set members, which is useful for recording the best known path cost. The value 0 identifies the blank. The goal is stored as a constant tuple.

Search data structures

- Frontier: a min-heap containing f-score, path cost, tie-breaker, and state.
- g_score: dictionary storing the best known path cost for each state.
- parent: dictionary storing the previous state and action for reconstruction.
- Serial counter: guarantees deterministic heap tie-breaking without comparing states.

When a better path to a state is found, the program updates its cost and parent and inserts a new heap entry. Older entries are skipped when removed if their stored path cost no longer matches the best known cost.

9. IMPLEMENTATION (CONTINUED)

Graphical user interface

The Tkinter window contains a nine-cell board, an input field, and controls for loading a state, loading a sample, shuffling, solving, stepping backward and forward, automatic playback, and reset. The blank cell is displayed with a muted background.

Execution flow

1. Launch the application with python main.py.
2. Enter nine digits, such as 123406758, and select Load State.
3. Select Solve with A* to calculate the optimal path.
4. Review the move count and search statistics.
5. Use Next and Previous to inspect individual states, or Auto Play to animate the solution.

The Shuffle function starts from the goal and performs a sequence of legal random moves. This guarantees that the generated state is solvable. It is preferable to random permutation because approximately half of all 8-puzzle permutations are unsolvable.

The application has no external package dependency. On some Linux installations Tkinter may need to be installed separately through the operating system package manager.

10. TEST CASES

ID	Input / Action	Expected Result	Observed
TC-01	Goal state 123456780	Accept; solution cost 0	Pass when observed
TC-02	Sample 123406758	Find optimal solution in 2 moves	Pass when observed
TC-03	Unsolvable 123456870	Reject using odd inversion parity	Pass when observed
TC-04	Duplicate digit 123456770	Display validation error	Pass when observed
TC-05	Too few digits 12345	Display validation error	Pass when observed
TC-06	Shuffle button	Generate a valid solvable state	Pass when observed
TC-07	Next / Previous	Move through solution states	Pass when observed
TC-08	Auto Play	Animate to the goal	Pass when observed

Important: execute each test on the submitted computer and replace “Pass when observed” with the actual result. Do not claim a test passed unless it has been run.

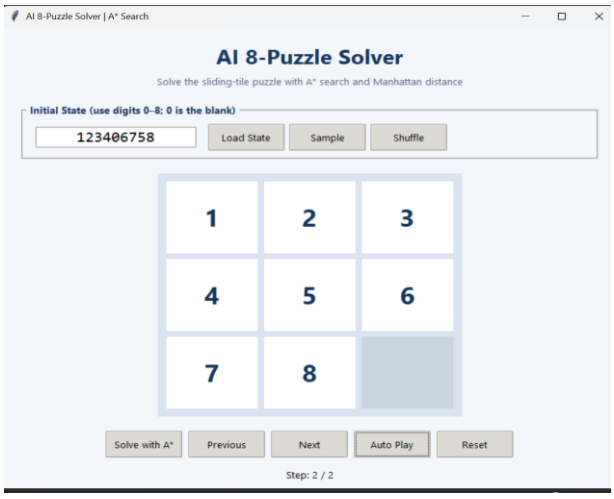
11. RESULTS AND SCREENSHOTS

For the sample initial state (1, 2, 3 / 4, blank, 6 / 7, 5, 8), the expected optimal solution is two moves: move the blank Down, then Right. The final board is the standard goal configuration. The application reports the measured execution time and the number of expanded and generated states.

Required screenshots (capture from your running application)

1. Application home screen showing the title, input field, and empty/sample board.
2. Initial state loaded in the 3×3 board.
3. A* search completed, with the performance statistics visible.
4. Intermediate state after the first move (blank moved Down).
5. Final goal state after the second move (blank moved Right).
6. Optional: unsolvable input warning or a shuffled puzzle being solved.





12. PERFORMANCE ANALYSIS

A* prioritizes states using the sum of the path cost and heuristic estimate. Manhattan Distance provides more guidance than an uninformed strategy because it estimates how far the numbered tiles are from their target cells. The algorithm maintains the best known path cost to avoid needlessly accepting a more expensive route to the same state.

Complexity

In the general case, A* can require exponential time and memory in the solution depth. Its practical performance depends on the branching factor, solution depth, and heuristic quality. The 8-Puzzle has at most four legal moves from a state, with fewer moves at edges and corners. The priority queue and dictionaries require memory proportional to the number of discovered states.

Measures displayed by the application

- Solution cost: number of moves in the optimal path.
- Expanded states: states removed from the frontier and expanded.
- Generated states: successor states considered during expansion.
- Execution time: elapsed wall-clock time around the search call.

Execution time is hardware- and operating-system-dependent. For a fair comparison, run multiple initial states on the same computer and report the exact states tested. Do not generalize a single sample result to every possible configuration.

13. ADVANTAGES AND LIMITATIONS

Advantages

- Uses an informed AI search strategy rather than random trial and error.
- Returns a shortest path for the standard unit-cost puzzle.
- Manhattan Distance is simple, admissible, and easy to explain.
- Input validation and inversion parity prevent invalid or impossible searches.
- The GUI makes the sequence of states understandable during demonstration.
- The implementation uses Python's standard library and is easy to run.

Limitations

- The application supports only the standard 3×3 8-Puzzle.
- A* stores many discovered states and can consume substantial memory.
- Hard instances may take noticeably longer than easy instances.
- The goal arrangement is fixed in the current version.
- The heuristic does not learn from data; this is a search-based AI project, not a machine-learning model.

14. FUTURE SCOPE

The project can be extended in several directions:

- Support configurable goal states and board dimensions.
- Add alternative heuristics such as misplaced-tile count and Linear Conflict.
- Compare A* with Breadth-First Search, Greedy Best-First Search, and Iterative Deepening A*.
- Add charts comparing expanded nodes, memory use, and execution time for benchmark states.
- Provide keyboard controls and a more polished visual theme.
- Export the solution path and performance summary to a text or CSV file.
- Extend the interface to a web application while keeping the search module reusable.

A larger 15-Puzzle could be explored as an advanced extension, although it requires stronger heuristics and more careful memory management. The current 8-Puzzle implementation provides a controlled foundation for such work.

15. CONCLUSION

The AI-Based 8-Puzzle Solver demonstrates how a classical search problem can be formulated and solved using Artificial Intelligence techniques. The project defines the initial state, goal state, legal actions, transition model, and unit step cost. It implements A* Search with Manhattan Distance and uses a priority queue to explore promising states.

The application validates input, checks solvability through inversion parity, reconstructs an optimal solution path, and displays the sequence through a Tkinter interface. The performance panel makes it possible to discuss the number of moves, expanded states, generated states, and execution time during evaluation.

The project meets the core learning objectives of state-space modelling, heuristic search, algorithm implementation, testing, and result visualization. Future work can add stronger heuristics, algorithm comparisons, and larger puzzle configurations.

16. REFERENCES

1. Russell, S. J., and Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
2. Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
3. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
4. Python Software Foundation. heapq — Heap queue algorithm. https://docs.python.org/3/library/heapq.html
5. Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
6. AIMA: Artificial Intelligence: A Modern Approach, online resources. https://aima.cs.berkeley.edu/

Appendix: submission checklist

- Replace all square-bracket placeholders with actual student and institution details.
- Run the program and execute the test cases.
- Capture at least 4–6 genuine screenshots and insert them in the Results chapter.
- Check the final page count and formatting after adding screenshots.
- Submit the source-code folder, dataset note (not applicable), report, and 8–10 slide presentation.